Assignment 05

Published | Assign To | Edit | ⋮

Learning Objectives

The objective of this assignment is to learn how to build custom machine images with our application and dependencies baked in.

Packer & Custom Images - Building Custom Application Images using Packer

For this assignment only, install `MySQL/MariaDB/PostgreSQL` locally in the custom image.

1. Use `Ubuntu 24.04 LTS` as your source image to create a custom image using Packer.
2. All custom images you build should be private. Only you can launch instances from it.
3. All custom image builds should happen in your `DEV` AWS account and must be automatically **shared** with your `demo` AWS account.
4. Custom image builds should be set up to run in your `default` VPC.
5. The custom image should include everything needed to run your application and the application binary itself. For e.g., if you are using Tomcat to run your Java web application, your custom image must have Java & Tomcat installed. You should also ensure the Tomcat service starts when an instance is launched. If you are using Python, make sure you have the right version of Python and that the libraries you need are installed in the custom image.
6. The packer template should be stored in the same repo as the web application.
7. ***For this assignment only**, install `MySQL/MariaDB/PostgreSQL` locally in the custom image.*

Packer Continuous Integration - Add New GitHub Actions Workflow for Status Check

When a pull request is raised, this GitHub Actions workflow should do the following:

1. Run the `packer fmt` command. If this command modifies the Packer template, the workflow should fail and prevent users from merging the pull request.
2. Run the `packer validate` command. If this command fails to validate the Packer template, the workflow should fail, and users should be prevented from merging the pull request.

Packer Continuous Integration - Add New GitHub Actions Workflow to Build Packer custom image

Make sure you do not trigger your workflow to build a custom image when a pull request is raised. It should only build a custom image when a pull request is merged.

Setup

1. You will need to create a new IAM service account (manually in the AWS console) in the `DEV` AWS for GitHub Actions and configure the security credentials in your organization's repository. Refer to **this (https://docs.github.com/en/actions/security-guides/using-secrets-in-github-actions)** for more details.
2. Grant the service account the roles needed. See the **documentation (https://github.com/hashicorp/packer-plugin-googlecompute/blob/main/docs/README.md#running-on-google-cloud)** for more details.
3. Install & configure `AWS cli` on your GitHub Actions runner with the marketplace action.

When a pull request is merged, this GitHub Actions workflow should be triggered to do the following:

1. Run the integration test.
2. Build the application artifact (war, jar, zip, etc.). This artifact should be built on the GitHub actions runner and not in the custom image, i.e., do not `git clone` your repository in the Packer template and then build it. You will build the artifact on the runner and copy it into the custom image.
3. Build the custom image in AWS with the following.
    1. Create a local user `csye6225` with the primary group `csye6225`. This user should not have a login shell, i.e. login shell should be `/usr/sbin/nologin`.
    See **https://www.baeldung.com/linux/create-non-login-user (https://www.baeldung.com/linux/create-non-login-user)** for details.
    2. Install application dependencies and set up the application by copying the application artifacts and the configuration files.
    3. The application artifacts, configuration, etc., must be owned by the user `csye6225` and group `csye6225`.
    4. Add your *systemd* service file to `/etc/systemd/system` and configure it to start your service when the instance is launched.
        1. Run `systemctl daemon-reload` with `sudo`
        2. Run `systemtl enable <service_name>` with `sudo`
4. No custom image should be built if any of the jobs or steps in the workflow fail.

Infrastructure as Code w/Terraform

In this assignment, you will update the Terraform template to add the following resources.

Application Security Group

1. Create an EC2 security group for your EC2 instances hosting web applications.
2. Add an ingress rule to allow TCP traffic on ports `22`, `80`, `443`, and the port on which your application runs from anywhere in the world.
3. This security group will be referred to as the `application security group`.

EC2 Instance

Create an EC2 instance with the following specifications. You may go with default values for any parameter not provided in the table below. The EC2 instance must be launched in the VPC created by Terraform. You cannot launch the EC2 instance in the `default` VPC.

- The application security group should be attached to this EC2 instance.
- Make sure the EBS volumes are terminated when EC2 instances are terminated.

Parameter	Value
Amazon Machine Image (AMI)	Your custom AMI
Protect against accidental termination	No
Root Volume Size	25
Root Volume Type	General Purpose SSD (GP2)

Assignment Submission Instructions

Development Workflow- Branch Management

- Complete all work on a feature branch in your forked repository
- Merge your feature branch to `main` only when development is complete
- Ensure your feature branch and `main` branch are synchronized before submission

Submission Requirements

Step 1: Create Submission Folder

Create a folder using this naming convention:

```
firstname_lastname_neuid_a_##
```

Replace `##` with the assignment number (e.g., `01`, `02`, etc.)

Step 2: Clone Repositories

1. Navigate to your submission folder
2. Clone **all** required GitHub repositories using:

bash

```
git clone [repository-url]
```

3. **Important:** Use `git clone` to preserve your complete commit history and branch information

Step 3: Create and Verify ZIP File

1. Create a ZIP file of your submission folder:

```
firstname_lastname_neuid_a_##.zip
```

2. **Verification Step:** Extract your ZIP file in a different location and confirm:
   - All repositories are present
   - Commit history is intact
   - All required branches exist

Step 4: Submit

Upload your ZIP file to the assignment submission portal.

Important Notes

- **Multiple Submissions Allowed:** You may resubmit if needed. Only your latest submission will be graded.
- **Quality Check:** Always verify your ZIP file contents before final submission to avoid issues.
- **Complete Repositories Required:** Submit code from all repositories mentioned in the assignment requirements.

Troubleshooting

If you encounter issues with your ZIP file, feel free to resubmit after verification. Take time to ensure all requirements are met before your final submission.

Grading Guidelines for Cloud-Native Web Application

System Stability & Performance

- **No Server Errors**: Application must not generate 500 internal server errors during testing
- **Application Uptime**: No restart of the application server required between API calls

- **Error Handling**: Application should handle invalid requests gracefully with appropriate HTTP status codes

Demo Requirements

- **Platform**: Students must demonstrate the web application from their own laptops
- **API Testing**: APIs must be tested using REST clients (Postman, Restlet, Insomnia, etc.)
  - Browser-based testing is **not acceptable**
- **UI Restriction**: Application must **not** have any user interface components
  - Verify complete absence of HTML pages, forms, or visual components

Security Implementation

Authentication & Authorization

- **Authentication Method**: Must use Basic Authentication with a token-based approach
  - Session-based authentication is **not allowed**
  - JWT tokens are **not allowed**
- **Password Security**: All passwords must be encrypted using BCrypt with salt
  - Verify in the database that passwords are properly hashed
  - Check that salt is unique per password

Data Protection

- **Response Payload Security**: Password fields must **never** be included in API responses
  - Test all endpoints that return user data
- **Username Validation**: Only valid email addresses can be used as usernames
  - Test rejection of non-email username formats

Data Integrity & Business Logic

Account Management

- **Duplicate Prevention**: Application must prevent the creation of multiple accounts with the same email address
  - Test duplicate email registration scenarios
  - Verify appropriate error responses for duplicate attempts
- **Audit Fields Protection**: System-managed fields must be read-only
  - `account_created` field: Set automatically by application, never by user input
  - `account_updated` field: Set automatically by application, never by user input
  - Test attempts to manually set these values through API calls

Code Quality & Standards

Configuration Management

- **No Hard-Coding**: Variable values must be configurable (environment variables, config files)
- **Default Values**: Acceptable when overrideable during runtime or demos

- **Demo Flexibility**: Values must be modifiable when requested by the instructor/TA

Code Style & Documentation

- Follow industry-standard style guides for your programming language
- Include meaningful comments for complex logic
- Maintain consistent formatting and naming conventions
- Provide clear README files with setup and usage instructions

Application Stability & Error Handling

Runtime Stability

- **No Crashes**: Application must handle API calls without crashing
- **No Restarts**: Application/VM should not require restarts during demos (unless specifically required for feature demonstration)
- **Graceful Degradation**: Handle edge cases and invalid inputs appropriately

Error Management

- **No 500 Errors**: All server errors must be properly handled
- **Meaningful Error Messages**: Provide clear, actionable error responses
- **Input Validation**: Validate and sanitize all user inputs
- **Resource Management**: Properly handle database connections, file operations, and external API calls

Edge Case Handling

- Test boundary conditions (empty inputs, maximum values, null cases)
- Handle network timeouts and connection failures
- Implement appropriate fallback mechanisms
- Validate data integrity throughout the application lifecycle

Repository Content & Hygiene

File Exclusion Standards

- **IDE Files**: No IDE-specific configuration files (`.idea/` , `.vscode/` , `.eclipse/` , etc.)
- **Build Artifacts**: No compiled files or build outputs
  - Java: No `.class` , `.jar` , `.war` files
  - Node.js: No `node_modules/` directory
  - General: No `build/` , `dist/` , `target/` directories
- **System Files**: No OS-specific files (`.DS_Store` , `Thumbs.db` , etc.)

Dependency Management

- **External Dependencies**: Third-party libraries from package managers (Maven, npm, pip) must **not** be committed
- **Gitignore Configuration**: Verify `.gitignore` file is properly configured and comprehensive
  - Must include language-specific patterns
  - Must exclude build artifacts and dependencies
  - Should include common IDE and OS files

Grading Guidelines

Warning: The following guidelines are for information only and are subject to change at the discretion of the instructor and TA.

Common Grading Rubrics

Git

1. All commits to GitHub repositories must be made via pull request. No changes can be made by pushing commits directly to the main branch. The exception to this is the first commit made to the repository to initialize it, which can be done directly from the UI.
2. All pull requests must have status checks and be enforced. Merging pull requests that did not pass status checks may result in penalties.

Security

1. Committing any kind of credentials to the code or uploading them as part of your Canvas submission may result in a penalty.
2. Service providers may automatically disable your keys or suspend your account if they detect keys being committed in your GitHub repositories, even if they are private. This can severely affect your assignment grade and your ability to work on assignments in the course. This is not grounds for assignment extensions; late penalties will apply as documented in the syllabus.

Software Engineering Best Practices

1. Values for variables should not be hardcoded in the code. It's OK to set default values for variables, but if the value cannot be modified during the demo or when the TA or instructor asks, you may be penalized.
2. Follow industry standards and use a style guide for your code. You can find them with simple web searches.

Code Stability, Error Handling, & Edge Cases

1. The application should not crash between various API calls.
2. The application, VM, or other resources should not require a restart during the demo unless the grading guidelines require it to demonstrate a specific feature/functionality.

3. The server cannot respond with a 500 Internal Server Error. This usually highlights areas of code that do not have proper error handling.

Previous Assignment Objectives

TAs must verify that students have resolved all open issues from the previous assignment(s).

Git & GitHub Repository (2%)

Git Repository Content Check

1. Check the repository for any IDE-specific files. IDE configuration files must not be in the repository.
2. Verify their .gitignore configuration.
3. Check the repository for build artifacts such as `.class`, `.jar`, `.war` files and `build`, `node_modules`, `venv` directory. None of these should be checked into the repository.
4. Check for dependencies. Dependencies from the Maven repository, npm, or Python should not be committed to the git repository.

Git Forking Workflow

1. No direct commits are made to the organization's repository. Verify that the student works from a forked repository and uses the feature branch. There should be no direct commits to the main branch in their forked repository.

IAM (8%)

1. The student has created a separate IAM user for GitHub Actions in their `DEV` account.
2. This user has a **custom IAM policy (https://developer.hashicorp.com/packer/integrations/hashicorp/amazon#iam-task-or-instance-role)** attached to them.
3. This IAM user does not have admin access to the AWS account.
4. This IAM user does not have console access.
5. This IAM user only has programmatic access to the account.

GitHub Action Workflow - Packer Status Check (20%)

1. The workflow checks to make sure the packer template is formatted correctly. Test this by raising a PR in which the template is not properly formatted.
2. The workflow validates the packer template. Test this by removing some closing brackets.
3. Students cannot merge the PR when any of the above checks fail.
4. This workflow does not build the AMI.

5. The workflow runs on the organization's repository and not in the forked repository.

GitHub Action Workflow - Packer AMI Build (20%)

1. This workflow runs only after the pull request is merged. It should not run when a pull request is raised.
2. This workflow builds an AMI in the `DEV` account and shares it with `DEMO` the account.
3. The AMI is private.
4. The AMI uses Ubuntu 24.04 LTS for the source image.
5. The Packer template does not contain any hard-coded values, so it's okay to set default values for variables.
6. The Packer template does not contain any AWS credentials.

Infrastructure as Code w/Terraform (40%)

1. `terraform apply` should create the networking, security group, and EC2 instance with the AMI built during the demo.
2. `terraform destroy` should delete all resources created in the above step.
3. Students **cannot** `ssh` to the instance to start their application. Test all APIs implemented in previous assignments, including the health check endpoint.
    1. Students cannot run `npm install` or `pip install` or any other commands to install dependencies, as they should have been installed and set up when the AMI was built.
4. For this assignment only, students will have a database running locally in the EC2 instance.
5. The database port, `3306` for `MySQL & MariaDB` and `5432` for `PostgreSQL` should not be included in the security group. No one can connect to the database from outside the virtual machine.
6. `git` should not be installed in the AMI. Check for it using the `which git` command.

Web Application on AWS (20%)

1. Launch the EC2 instance from the console from the machine image created with Packer.
2. The web application API endpoints should be accessible from the internet and work as expected. Ask students to SSH into the VM in AWS and show the data in the database before and after the test.

Points	10
Submitting	a file upload

File Types	zip
Allowed Attempts	3

Due	For	Available from	Until
Oct 16 at 3pm	Everyone	Oct 9 at 6pm	Oct 17 at 3pm

+ Rubric